

5/06/2025

Recurrent Neural Networks (RNNs) (State Space Model)

Is an evolution of FNN in which you allow loops in the graph that defines the computational structure.

This is possible because you change the neural computation as follows.

FNNs

$$x_i = \sigma \left(\sum_{j \in \text{pa}(i)} W_{ij} x_j \right)$$

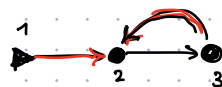
RNN

you add another index t ("time")

$$x_i(t+1) = \sigma \left(\sum_{j \in \text{pa}(i)} W_{ij} x_j(t) \right)$$

And you assign $x_i(0)$ for all neurons

Ex 1



$$x_1(0) = u(0)$$

$$x_2(0) = \frac{1}{2}, \quad x_3(0) = 1 \quad \parallel \text{ Usually it is a random initialization or you initialize to 0.}$$

$$x_1(1) = u(1) \text{ given value!}$$

$$x_2(1) = \sigma \left(W_{21} x_1(0) + W_{23} x_3(0) \right)$$

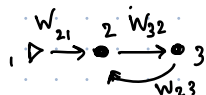
" $u(0)$ "

$$\text{pa}(2) = \{1, 3\}$$

$$\dot{x} = f(x) \quad \text{ODE}$$

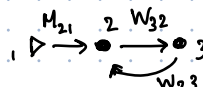
Let's talk about inputs

i) $x_i(t+1) = \sigma \left(\sum_{j \in \text{pa}(i)} W_{ij} x_j(t) \right) \leftarrow$ In this representation x_j can be a proper neuron or an input to the network.



ii) $x_i(t+1) = \sigma \left(\sum_j W_{ij} x_j(t) + \sum_j M_{ij} u_j(t) \right)$

" $u_j(t)$ " ↑ inputs



ii) - bias

$$z_i(t+1) = \sigma \left(\sum_j W_{ij} x_j(t) + \sum_j M_{ij} u_j(t+1) \right)$$

$\xrightarrow{M_{21}} 2 \xrightarrow{W_{32}} 3$
 $\xleftarrow{W_{23}}$

inputs

Graphical representation

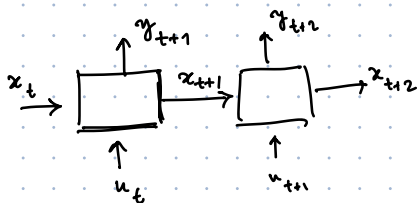
With z you mean the collection of all neuron outputs $z = (z_1, z_2, \dots)$

Many times the notation h is used in place of z (Y. Bengio & others book)

The quantity z (or h) is usually referred to as "state".

Beside the state you typically have

1. Inputs (u)
2. Outputs (y)



z_{t+1} is computed according to i, ii or iii - h's above

$u_1, u_2, u_3, \dots$ is given and represent the input to the network

$y_{t+1} = f(z_{t+1})$ for instance a common choice is $y_{t+1} = \sigma_{out}(C z_{t+1} + D)$

The simplest case is the one in which $r_{out} = id$ $D = 0$

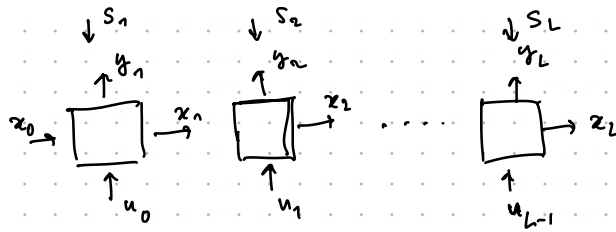
C select (project) some components of z_{t+1}

Learning Recurrent Neural Networks

Learning is done on datasets of sequences. (u^i, s^i) ↗ supervised learning

$$u^i = u_1^i, u_2^i, \dots, u_L^i$$

$$s^i = \text{supervision}$$



$y_1, \dots, y_L$ is the sequence of outputs
 $s_1, \dots, s_L$ is the sequence of supervisions

$$e^i = \frac{1}{L} \sum_{t=1}^L \ell(y_t^i, s_t^i) \quad i \text{ labels an example in the training set.}$$

$$E = \frac{1}{L} \frac{1}{N} \sum_{t=1}^L \ell(y_t^i, s_t^i)$$

BPTT Back propagation through time

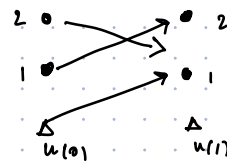
It is standard backpropagation applied to the unfolded network.



$t=0$
 $u(0)$ is given
 $x_1(0)$ is given
 $z_2(0)$ is given



$t=1$
 $u(1)$ is given
 $x_1(1) = \sigma(W_{10} u(0) + W_{12} x_2(0))$
 $x_2(1) = \sigma(W_{21} x_1(0))$



$t=2$

$u(2)$ is given

$$x_1(2) = \sigma(W_{10} u(1) + W_{12} x_2(1))$$

$$x_2(2) = \sigma(W_{21} x_1(1))$$

$$x_1(2) = \sigma(W_{10} u(1) + W_{12} \sigma(W_{21} x_1(0)))$$

$$x_2(2) = \sigma(W_{21} \sigma(W_{10} u(1) + W_{12} x_2(0)))$$

$\rightarrow$ we know their dependence on the weights

Real Time Recurrent Learning